COMP2300 Set 8 Practice Exam

Topics: ILP, Dependences, Dynamic Scheduling, Renaming, ROB, Speculation, LSQ

Time: 120 minutes **Total:** 100 marks

Instructions

- Answer in English.
- Part A is multiple choice.
- Part B contains dependence analysis and out-of-order reasoning.
- Part C contains adapted past-exam style questions with source tags.

Part A: Multiple Choice (30 marks, 3 marks each)

Choose exactly one option for each question.

A1. Which dependence corresponds to a `RAW` hazard?

(A) Anti dependence
(B) Output dependence
(C) True dependence
(D) Control dependence

A2. Register renaming primarily eliminates:

(A) RAW only
(B) WAR and WAW
(C) Branch hazards only
(D) Cache misses

A3. An issue queue is used to:

(A) Store only committed results
(B) Hold waiting instructions until operands are ready
(C) Replace the branch predictor
(D) Encode immediate values

A4. Which statement about `ROB` is correct?

(A) It guarantees that instructions execute in order
(B) It supports in-order retirement of speculative results

(C) It stores branch target addresses only

(D) It removes all RAW hazards

A5. A precise exception requires that:

(A) Younger instructions may already be permanently committed

(B) Older instructions after the faulting instruction may commit first

(C) All older instructions have completed and the faulting instruction has not improperly committed

(D) The CPU flushes only the fetch stage

A6. Which dependence cannot be removed by renaming?

(A) WAR

(B) WAW

(C) RAW

(D) Output name dependence

A7. In a modern OOO design, speculative results are typically written first to:

(A) ARF only

(B) ROB

(C) Instruction memory

(D) BTB

A8. The main limitation of a scoreboard-only design highlighted in the lecture is:

(A) It has no ALUs

(B) It cannot support any form of data dependence

(C) It struggles with false dependences and precise recovery

(D) It eliminates branch hazards completely

A9. A load-store queue is used because:

(A) Loads and stores are always safe to reorder

(B) Memory addresses may alias, so load/store reordering needs checking

(C) It replaces all caches

(D) It is the same thing as a branch history table

A10. Memory-level parallelism refers to:

(A) Using one memory request at a time very quickly

(B) Exploiting multiple memory requests in flight concurrently

(C) Replacing DRAM with SRAM

(D) Turning branches into arithmetic

Part B: Computational / Reasoning Problems (50 marks)

B1. Classifying dependences (8 marks)
For each pair below, classify the dependence as **RAW**, **WAR**, **WAW**, or **none**.

```
i1: LDR R2, [R6, #0]
i2: ADD R3, R2, R5

i3: ADD R6, R3, R5
i4: LDR R2, [R6, #0]

i5: ADD R2, R7, R8
i6: SUB R2, R9, R10
```

B2. Renaming to remove false dependences (8 marks)
Rename the following code using temporary names `T0` and `T1` so that false dependences are removed while true dependences are preserved.

```
i1: ADD R1, R2, R3
i2: SUB R2, R4, R5
i3: ADD R6, R1, R7
i4: MUL R2, R8, R9
```

State which original false dependence(s) you removed.

B3. ROB allocation and retirement order (8 marks)
Suppose three instructions enter rename in program order and are assigned tags `ROB0`, `ROB1`, and `ROB2`.

(a) Which tag belongs to the oldest instruction?

(b) If `ROB2` finishes execution first, may it retire first? Why or why not?

(c) Which instruction must be at the head of the ROB before `ROB2` may retire?

B4. Speculation and precise exceptions (8 marks)
An older load instruction causes an exception. Two younger arithmetic instructions have already finished execution out of order.

(a) In a design that writes results directly into the architectural register file, why can this become an imprecise exception?

(b) How does a ROB-based design avoid this problem?

B5. Load/store reordering (8 marks)
For each case below, state whether reordering is safe and briefly justify:

(a) An older store writes address `A`; a younger load reads address `B`, where `A` and `B` are known to be different.

(b) An older store writes address `A`; a younger load reads address `A`.

(c) The address of the older store is not yet known.

B6. OOO overlap on a cache miss (10 marks)
A load instruction misses in cache and waits many cycles for memory. Behind it are:

- one dependent ADD
- two independent arithmetic instructions
- one younger branch with operands already ready

(a) Which instruction must wait because of a true dependence?

(b) Which younger instructions may still issue in an OOO design?

(c) Why is this impossible in a strict in-order design?

Part C: Past Exam Questions (Source-Tagged) (20 marks)

C1. Register renaming (Source: exam24r.pdf, Q7b; 2023_Final_Exam.pdf, Q2 rename topic, adapted) (7 marks)
Explain how register renaming removes false dependences while preserving true dependences. Why does this help out-of-order execution?

C2. OOO pipeline issues addressed by ARF+ROB (Source: exam2025-main.pdf, Q3C; 2023_Final_Exam_SuppDA.pdf, ARF+ROB discussion, adapted) (7 marks)
List four problems in a scoreboard-style out-of-order pipeline that are addressed by adding register renaming and hardware speculation.

C3. Issue queue and dynamic scheduling (Source: exam2025-main.pdf, Q3; exam24r.pdf, Q7c, adapted) (6 marks)
Briefly explain the role of the issue queue in a dynamically scheduled processor. Why does it help tolerate long-latency misses better than a simple in-order pipeline?